

# MLOps Fundamentals

Introduction • ML Lifecycle • Environment Setup • Model Serialization & Configuration  
Management

*A practical reference guide*

# 1. Introduction to MLOps

## 1.1 What is MLOps?

MLOps (Machine Learning Operations) is the discipline of applying DevOps principles — automation, continuous integration, continuous delivery, monitoring, and collaboration — to the development and operation of machine learning systems. It bridges the gap between data science (experimentation, model building) and engineering (production systems, reliability, scale).

Where traditional software delivers deterministic code, ML systems ship a combination of code, data, and a trained model — three things that all change independently and all need to be versioned, tested, and monitored. MLOps exists because a model that performs well in a notebook frequently fails in production due to data drift, environment mismatches, or lack of monitoring.

## 1.2 Why MLOps Matters

- Reproducibility — the ability to recreate a model's results given the same data, code, and configuration
- Reliability — catching failures (data drift, model degradation, pipeline breaks) before they impact users
- Velocity — shortening the time from a trained model to a deployed, monitored service
- Collaboration — giving data scientists, ML engineers, and operations teams a shared, auditable workflow
- Governance & compliance — traceability of what data and model version produced a given prediction, which matters heavily in regulated industries (finance, healthcare)

## 1.3 MLOps vs. DevOps

| Dimension          | DevOps                   | MLOps                                                           |
|--------------------|--------------------------|-----------------------------------------------------------------|
| What changes       | Code                     | Code, data, and model — three moving parts                      |
| Testing            | Unit / integration tests | Also requires data validation and model quality tests           |
| Versioning         | Source code (Git)        | Code + data (DVC/LakeFS) + model artifacts + experiment configs |
| Deployment trigger | Code merge               | Code merge, new data, or model performance decay                |
| Monitoring focus   | Uptime, latency, errors  | Also data drift, concept drift, prediction quality              |
| Rollback           | Previous code build      | Previous model version and/or previous data snapshot            |

## 1.4 MLOps Maturity Levels

A widely used framework (adapted from Google Cloud) describes three levels of MLOps maturity:

## Level 0 — Manual Process

- Data preparation, training, and evaluation are manual, script-driven, and disconnected
- Deployment is a manual hand-off from data scientist to engineering
- No systematic monitoring after deployment; retraining happens rarely and manually

## Level 1 — ML Pipeline Automation

- Training becomes a repeatable, automated pipeline (data validation → training → evaluation)
- Continuous training is possible when new data arrives
- Still requires manual deployment of the pipeline itself

## Level 2 — CI/CD Pipeline Automation

- A full CI/CD system builds, tests, and deploys both the training pipeline and the serving pipeline automatically
- Includes automated data/model validation gates before promotion to production
- Supports rapid, low-risk iteration — the target state for mature ML organizations

## 1.5 Core Components of an MLOps System

- Version control — for code (Git), data (DVC, LakeFS, Delta Lake), and models (MLflow Model Registry, SageMaker Model Registry)
- Experiment tracking — logging parameters, metrics, and artifacts for every training run (MLflow, Weights & Biases, Neptune)
- Pipeline orchestration — automating multi-step workflows (Kubeflow Pipelines, Airflow, Vertex AI Pipelines, SageMaker Pipelines)
- Model serving — exposing a trained model as an API or batch job (TorchServe, TensorFlow Serving, KServe, SageMaker Endpoints)
- Monitoring — tracking latency, throughput, data drift, and prediction quality in production
- CI/CD — automated testing and promotion of pipelines, data validation logic, and serving code

## 2. The ML Lifecycle

The ML lifecycle describes the end-to-end journey of a machine learning system, from an initial business question to a monitored production service. Unlike traditional software, this lifecycle is cyclical — production monitoring routinely feeds back into retraining.

### 1. Problem Definition

Translate a business question into a measurable ML objective (e.g., 'reduce claims fraud false negatives by 15%'). Define success metrics, constraints (latency, interpretability, fairness), and what a baseline (non-ML) solution would achieve.

### 2. Data Collection

Identify and gather data sources — databases, event streams, third-party feeds, logs. Establish data contracts and access/compliance requirements early, especially for regulated data (PII, PHI).

### 3. Data Preparation & Feature Engineering

Clean, transform, and validate raw data. Handle missing values, outliers, and class imbalance. Engineer features and build a feature store when features are reused across models.

### 4. Model Development & Training

Select candidate algorithms, train models, and tune hyperparameters. Track every run's parameters, metrics, and artifacts so experiments are comparable and reproducible.

### 5. Model Evaluation

Assess the model against held-out data using the metrics defined in step 1, alongside fairness, robustness, and error-analysis checks. Compare against the baseline before considering deployment.

### 6. Model Packaging & Serialization

Serialize the trained model into a portable format, bundle it with its dependencies and configuration, and version it in a model registry.

### 7. Deployment

Release the model behind an API (online serving), as a batch job, or embedded on-device. Common patterns include shadow deployment, canary release, and blue/green rollout to limit risk.

### 8. Monitoring

Track system health (latency, errors) and ML-specific health (data drift, concept drift, prediction distribution, accuracy where ground truth becomes available).

### 9. Retraining & Iteration

Use monitoring signals and new data to trigger retraining, either on a schedule or automatically when drift crosses a threshold — closing the loop back to step 3 or 4.

**Key idea:** *The ML lifecycle is a loop, not a line. Production monitoring is not the end of the process — it is the trigger for the next iteration of data preparation and training.*

## 3. Environment Setup

Reproducible environments are foundational to MLOps. A model trained in one environment must behave identically when retrained or served in another — across a laptop, a CI runner, and a production cluster.

### 3.1 Local Development Environments

- Python virtual environments (venv, virtualenv) isolate project dependencies from the system Python installation
- Conda/Mamba environments additionally manage non-Python dependencies (CUDA libraries, compiled packages) — common in deep learning workflows
- Poetry or pip-tools lock exact dependency versions for reproducible installs across machines

Example: creating an isolated environment

```
# Using venv
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Using conda
conda create -n mlops-env python=3.11
conda activate mlops-env
conda env export > environment.yml
```

### 3.2 Dependency Management

- requirements.txt — simple pinned dependency list for pip-based projects
- pyproject.toml (Poetry/PDM) — declares dependencies plus a lock file for fully reproducible installs
- environment.yml — Conda's equivalent, capturing both Python and system-level packages

A good practice is to pin exact versions (e.g., scikit-learn==1.4.2) rather than ranges, since minor version changes in ML libraries can silently change model behavior.

### 3.3 Containerization

Docker packages the application code, its dependencies, and the runtime into a single portable image — eliminating ‘works on my machine’ issues and making the same artifact deployable locally, in CI, and in production.

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src
COPY model/ ./model
```

```
ENV MODEL_PATH=/app/model/model.pkl  
EXPOSE 8080
```

```
CMD ["python", "src/serve.py"]
```

- Use multi-stage builds to keep final images small (build dependencies in one stage, copy only artifacts to the runtime stage)
- Pin the base image tag (e.g., python:3.11-slim, not python:latest) for reproducibility
- For GPU workloads, base images typically extend NVIDIA CUDA images (e.g., nvidia/cuda) with the appropriate driver/toolkit versions

### 3.4 Managed Cloud ML Environments

| Platform | Managed environment service                                       |
|----------|-------------------------------------------------------------------|
| AWS      | SageMaker (Studio notebooks, managed training/serving containers) |
| Azure    | Azure Machine Learning (compute instances, curated environments)  |
| GCP      | Vertex AI (Workbench notebooks, custom training containers)       |

These platforms provide pre-built, versioned containers for common frameworks (PyTorch, TensorFlow, scikit-learn) and manage GPU provisioning, reducing environment-setup overhead for training and serving.

### 3.5 Reproducibility Checklist

- Pin language, framework, and library versions explicitly
- Fix random seeds for data splits, initialization, and any stochastic training steps
- Version the training data itself (not just the code) — e.g., with DVC or a data lake snapshot/tag
- Capture hardware-relevant details (GPU type, CUDA/cuDNN version) when they affect numerical results
- Store the exact environment definition (Dockerfile, lock file) alongside the model artifact

## 4. Model Serialization & Configuration Management

### 4.1 What is Model Serialization?

Serialization is the process of converting a trained model's in-memory state (weights, architecture, learned parameters) into a file format that can be saved, transferred, and reloaded later — in a different process, machine, or even a different programming language.

### 4.2 Common Serialization Formats

| Format                | Typical use case                            | Notes                                                                                                                    |
|-----------------------|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Pickle (.pkl)         | scikit-learn and general Python objects     | Python-specific; not safe to load untrusted files; not cross-language                                                    |
| Joblib                | scikit-learn models with large NumPy arrays | More efficient than pickle for numeric arrays; still Python-only                                                         |
| ONNX                  | Cross-framework interoperability            | Exports models from PyTorch, TensorFlow, scikit-learn into one shared format for inference across runtimes/languages     |
| TensorFlow SavedModel | TensorFlow / Keras models                   | Includes the computation graph and weights; works well with TF Serving                                                   |
| TorchScript           | PyTorch models                              | Compiles a PyTorch model into a serializable, optimizable graph for C++/production runtimes                              |
| Safetensors           | Large models, especially LLMs               | Faster and safer than pickle (no arbitrary code execution on load); increasingly the default for transformer checkpoints |
| PMML                  | Traditional statistical/ML models           | XML-based, vendor-neutral; common in legacy enterprise/BI tooling                                                        |

### 4.3 Choosing a Serialization Format

- Staying within one framework and one language → the framework's native format (SavedModel, TorchScript) is usually simplest
- Need to serve across languages or runtimes (e.g., train in Python, serve in Java/C++) → ONNX is designed for this
- Working with large models or untrusted model sources → prefer safetensors over pickle, since pickle can execute arbitrary code on load
- Working in a regulated, tool-agnostic enterprise setting → PMML may still be required by legacy scoring engines

## Example: saving and loading with joblib and ONNX

```
import joblib
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier().fit(X_train, y_train)
joblib.dump(model, 'model.joblib')

loaded_model = joblib.load('model.joblib')

# Exporting to ONNX for cross-platform serving
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

onnx_model = convert_sklearn(
    model,
    initial_types=[('input', FloatTensorType([None, X_train.shape[1]]))]
)
with open('model.onnx', 'wb') as f:
    f.write(onnx_model.SerializeToString())
```

## 4.4 Model Metadata & Registries

A serialized model file alone is not enough for production use — it needs metadata describing how it was produced and how it should be used. A model registry (MLflow Model Registry, SageMaker Model Registry, Vertex AI Model Registry) typically tracks:

- Training data version/snapshot reference
- Hyperparameters and training configuration used
- Evaluation metrics and the environment (library versions) the model was trained in
- Lifecycle stage — e.g., Staging, Production, Archived — and approval/audit history

## 4.5 Configuration Management

Configuration management separates the things that change between environments or experiments — hyperparameters, file paths, feature flags, resource limits — from the code itself, so the same codebase can run in dev, staging, and production without modification.

### 4.5.1 Configuration as Code (YAML/JSON)

Most ML projects externalize configuration into YAML or JSON files rather than hardcoding values in scripts:

```
# config/train.yaml
model:
  type: random_forest
  n_estimators: 200
  max_depth: 12

data:
```

```
train_path: s3://ml-bucket/data/train_v3.parquet
test_size: 0.2
random_seed: 42
```

```
training:
  cross_validation_folds: 5
  early_stopping_rounds: 20
```

### 4.5.2 Hierarchical Configuration with Hydra

Tools like Hydra (Python) let you compose configuration from multiple files and override any value from the command line, which is useful for running many experiment variants without duplicating config files:

```
python train.py model.n_estimators=500 data.random_seed=7
```

### 4.5.3 Environment Variables & Secrets

- Non-sensitive, environment-specific values (endpoints, log levels) are commonly injected via environment variables
- Sensitive values (API keys, database credentials) belong in a secrets manager (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault) — never committed to config files in version control

### 4.5.4 Versioning Configuration Alongside Code and Data

- Commit configuration files to Git so every training run is tied to a specific config version
- Log the resolved configuration (after any command-line overrides) as part of experiment tracking, so a past run can be exactly reproduced
- Use tools like DVC to version large config-adjacent artifacts (e.g., feature lists, preprocessing pipelines) that don't belong well in Git directly

**Best practice:** *Treat configuration with the same rigor as code: review changes via pull request, keep it out of hardcoded scripts, and always log the exact configuration used for every training run and every deployed model.*

## Summary

MLOps applies engineering discipline to the full ML lifecycle — from problem definition through retraining — so that models are not just accurate in a notebook, but reproducible, monitorable, and safe to operate in production. A solid environment setup (isolated, containerized, version-pinned) and disciplined model serialization and configuration management are the foundation that everything else in MLOps — CI/CD, monitoring, and automated retraining — is built on top of.